

examples of valid identifiers are

AvgTemp	count	a4	\$test	this_is_ok
---------	-------	----	--------	------------

Invalid identifier names include these:

2count	high-temp	Not/ok
--------	-----------	--------

NOTE Beginning with JDK 8, the use of an underscore by itself as an identifier is not recommended.

Literals

A constant value in Java is created by using a *literal* representation of it. For example, here are some literals:

100	98.6	'X'	"This is a test"
-----	------	-----	------------------

Left to right, the first literal specifies an integer, the next is a floating-point value, the third is a character constant, and the last is a string. A literal can be used anywhere a value of its type is allowed.

There are three types of comments defined by Java. You have already seen two: single-line and multiline. The third type is called a documentation comment. This type of comment is used to produce an HTML file that documents your program. The Chapter 2 An Overview of Java 33 Part I documentation comment begins with a `/**` and ends with a `*/`.

```
/**
 * This is a documentation comment for a class.
 * It provides an overview of the class's purpose and functionality.
 * You can include various Javadoc tags here to provide more structured information.
 * @author Your Name
 * @version 1.0
 * @since 2025-11-06
 */
public class MyClass {

    /**
     * This is a documentation comment for a field.
     * It explains the purpose of the field.
     */
    private int myField;

    /**
     * This is a documentation comment for a method.
     * It describes what the method does, its parameters, and its return value.
     * @param parameter1 A description of the first parameter.
     * @param parameter2 A description of the second parameter.
     * @return A description of the value returned by the method.
     * @throws IllegalArgumentException if an invalid argument is provided.
     */
}
```

```

public int myMethod(int parameter1, String parameter2) throws IllegalArgumentException
{
    // Method implementation
    return 0;
}
}

```

In Java, there are a few characters that are used as separators. The most commonly used separator in Java is the semicolon. As you have seen, it is used to terminate statements. The separators are shown in the following table:

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.
::	Colons	Used to create a method or constructor reference. (Added by JDK 8.)

In addition to the keywords, Java reserves the following: `true`, `false`, and `null`. These are values defined by Java. You may not use these words for the names of variables, classes, and so on.

`System.out`. `System` is a class predefined by Java that is automatically included in your programs.

`System.out.println()` is actually **not a special keyword**, but a **chain of objects and classes working together**.

Let's break it visually and intuitively 📌

The line:

```
System.out.println("Hello, world!");
```

Looks simple, but inside it's actually:

Class → Object → Method

or in layers:

System → out → println()

Step-by-Step Explanation

1 System

- It's a **built-in class** in the `java.lang` package.
- It's **automatically imported** — so you don't need to write `import java.lang.System;`
- The `System` class provides access to **system-level resources**:
 - `System.out` → standard output stream (prints to console)
 - `System.in` → standard input stream (reads from keyboard)
 - `System.err` → standard error stream (prints error messages)

So when your book says:

“System is a class predefined by Java that is automatically included”

It means this class is already available — you don't have to define or import it.

2 out

- It's a **static field (variable)** inside the `System` class.
- It is an object of **PrintStream** class.

- It represents the **standard output stream** — where your printed text goes (the console).

So:

System.out

→ means “the output stream of the System class.”

3 println()

- It’s a **method** inside the `PrintStream` class.
- It means “print the given value and move to the next line.”

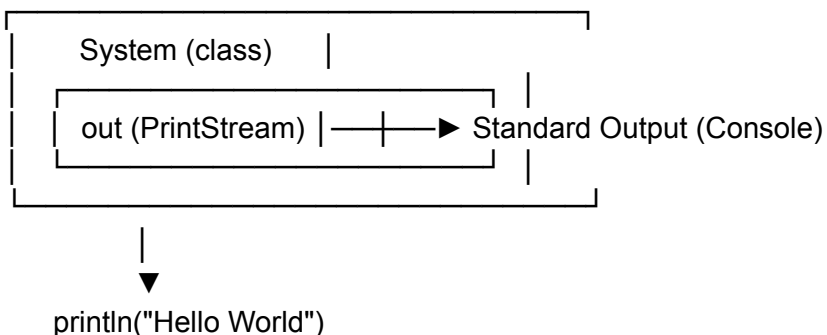
So:

System.out.println("Hello");

means:

“Use the System class → access its output stream → call the println method on that stream → print text to console.”

Visually:



In short:

Part	Type	Meaning
<code>System</code>	Class	Predefined in <code>java.lang</code>
<code>out</code>	Static object (of <code>PrintStream</code>)	Standard output stream
<code>println()</code>	Method	Prints text + newline

Example Variations:

`System.out.print("Hello");` // prints on same line

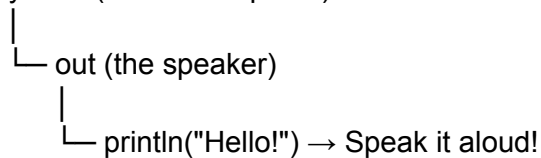
`System.out.println("World");` // prints + new line

`System.err.println("Error!");` // prints to error stream (in red on some consoles)

Analogy time

Imagine this like sending a message to a speaker system:

System (the control panel)



So:

- `System` = control box
 - `out` = speaker connected to it
 - `println()` = button you press to say something through it
-

Java is a strongly typed language. Java's safety and robustness comes from this fact. First, every variable has a type, every expression has a type, and every type is strictly defined.

Second, all assignments, whether explicit or via parameter passing in method calls, are checked for type compatibility. There are no automatic coercions or conversions of conflicting types as in some languages. The Java compiler checks all expressions and parameters to ensure that the types are compatible. Any type mismatches are errors that must be corrected before the compiler will finish compiling the class.

1 “Strongly Typed” — What It Means

When a language is **strongly typed**, it means:

- Every variable and expression has a **defined data type**, and
- You **cannot mix types** in invalid or unsafe ways.

The compiler will **check and enforce** that all type usages are consistent — before the program even runs.

So in Java:

```
int x = 5;
```

```
x = "Hello"; // ❌ ERROR: Can't assign String to int
```

The compiler stops you *right here*.

It doesn't let your program run with potential type confusion.

2 Java's “Type Safety”

In Java:

- Every **variable** has a specific type (`int`, `String`, `boolean`, etc.).
- Every **expression** has a specific type (e.g., `x + 2` is `int` if `x` is `int`).
- Every **method** has a declared return type.
- You **can't bypass** type rules unless you explicitly cast (and even then, it's checked).

This ensures **safety and robustness**, meaning:

- Fewer runtime errors.

- Fewer memory access issues.
 - More predictable behavior.
-

Visual View

+-----+-----+		
Data Type	What It Stores	
+-----+-----+		
int	Whole numbers (1, 2, 3)	
double	Decimal numbers (1.5)	
char	Single characters ('A')	
String	Text ("Hello")	
boolean	true / false	
+-----+-----+		

Now, Java won't let you "mix" these up.

For example:

```
int a = 10;
```

```
String b = "20";
```

```
int c = a + b; // ❌ Compile-time error
```

You must explicitly convert:

```
int c = a + Integer.parseInt(b); // ✅
```

3 "Every expression has a type"

That means even *operations* result in a specific type:

```
int a = 5;
```

```
double b = 2.5;
```

```
double c = a + b; // result is double
```

The compiler decides that:

- $a \rightarrow \text{int}$
- $b \rightarrow \text{double}$
- $a + b \rightarrow \text{double}$ (automatic type promotion)

4 Java vs Weakly Typed Languages

Feature	Java (Strongly Typed)	JavaScript (Weakly Typed)
Variable types fixed	✅ Yes	❌ No
Type checking	At compile time	At runtime
Example	<pre>int a = 5; a = "hi"; // ❌ Error</pre>	<pre>let a = 5; a = "hi"; // ✅ works (but risky)</pre>
Safety	High	Low
Speed	Faster at runtime	May slow down (runtime type checks)

5 Why “Strong Typing” Makes Java Safe

Because the compiler **catches type errors early** — before your code runs.

So Java avoids:

- **Type confusion** (e.g., adding a string to a number by accident)
- **Invalid memory access** (since types determine how data is stored)
- **Unexpected crashes**

That’s why your book says:

“Java’s safety and robustness comes from this fact.”

Analogy

Think of **types** as “labels” on storage boxes in a warehouse:

 Box 1 — int only

 Box 2 — String only

 Box 3 — boolean only

Java is the strict warehouse manager:

“You can’t put a banana (String) in the tools box (int)!”

This strictness prevents mess and confusion later.

In short:

Java being strongly typed means that **every variable and expression has a well-defined data type**, and the compiler **enforces these types** — preventing you from mixing them up, which makes programs **safer, more reliable, and less error-prone**.

The Primitive Types

Java defines eight *primitive* types of data: **byte**, **short**, **int**, **long**, **char**, **float**, **double**, and **boolean**. The primitive types are also commonly referred to as *simple* types, and both terms will be used in this book. These can be put in four groups:

- **Integers** This group includes **byte**, **short**, **int**, and **long**, which are for whole-valued signed numbers.
- **Floating-point numbers** This group includes **float** and **double**, which represent numbers with fractional precision.
- **Characters** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- **Boolean** This group includes **boolean**, which is a special type for representing true/false values.

The primitive types represent single values—not complex objects. Although Java is otherwise completely object-oriented, the primitive types are not. They are analogous to the simple types found in most other non-object-oriented languages. The reason for this is efficiency. Making the primitive types into objects would have degraded performance too much. The primitive types are defined to have an explicit range and mathematical behavior. Languages such as C and C++ allow the size of an integer to vary based upon the dictates of the execution environment. However, Java is different. Because of Java's portability requirement, all data types have a strictly defined range. For example, an **int** is always 32 bits, regardless of the particular platform. This allows programs to be written that are guaranteed to run without porting on any machine architecture. While strictly specifying the size of an integer may cause a small loss of performance in some environments, it is necessary in order to achieve portability.



1 What It Means That “Java Is Object-Oriented”

In an **object-oriented language**,

→ Everything is supposed to be an **object** — that is, an *instance of a class*.

Objects have:

- **State (data)** → fields or attributes
- **Behavior (methods)** → functions that act on that data

Example:

```
String s = "Hello";
```

```
System.out.println(s.length()); // calling a method on an object
```

Here, `s` is an **object** of class `String`,
and `.length()` is a method defined inside that class.

So far, so object-oriented ✅

💡 2 But Then... We Have Primitives 😞

Java has 8 **primitive types**, and they're *not objects*.

Category	Type	Example
Integer types	<code>byte</code> , <code>short</code> , <code>int</code> , <code>long</code>	<code>int a = 5;</code>
Floating point	<code>float</code> , <code>double</code>	<code>double d = 3.14;</code>
Character	<code>char</code>	<code>'A'</code>
Boolean	<code>boolean</code>	<code>true</code>

These are **raw, low-level** data types — stored directly in memory, not as object references.

They don't come from any class like `java.lang.Integer` or `java.lang.Boolean`.
And you can't call methods on them directly.

For example:

```
int x = 10;
```

```
// x.length(); ❌ Error — primitives have no methods!
```

That's why the book says:

“Although Java is otherwise completely object-oriented, the primitive types are not.”

3 Why Java Made This Exception

Java designers **intentionally** made primitive types non-object-oriented for **performance reasons**.

Objects in Java live on the **heap** and need memory allocation + garbage collection.
But primitives are stored directly in the **stack** — super fast to create, copy, and destroy.

So primitives make your code more efficient.

Example:

```
int a = 5; // stored directly
```

```
Integer b = 5; // object wrapping the value 5
```

The second one is slower and uses more memory because it creates an actual object.

4 The Bridge Between Both Worlds — Wrapper Classes

To make primitives “fit” into the object world when needed, Java provides **wrapper classes** in `java.lang` package:

Primitive	Wrapper Class
-----------	---------------

<code>byte</code>	<code>Byte</code>
-------------------	-------------------

<code>short</code>	<code>Short</code>
--------------------	--------------------

<code>int</code>	<code>Integer</code>
------------------	----------------------

`long` `Long`

`float` `Float`

`double` `Double`

`char` `Character`

`boolean` `Boolean`

Example:

```
int x = 10;
```

```
Integer y = Integer.valueOf(x); // wraps primitive in an object
```

Now you can call methods:

```
System.out.println(y.toString()); //  works, because y is an object
```

5 Auto-Boxing and Unboxing (Java's Magic Trick)

Java later introduced **autoboxing** to make the switch seamless:

```
Integer obj = 10; // auto-boxing: int -> Integer
```

```
int val = obj;    // auto-unboxing: Integer -> int
```

You write fewer conversions, but under the hood, Java still wraps and unwraps.

6 Visualization

Primitive (not object)

[int x = 5] ---> | 5 |

Wrapper (object)

[Integer y = 5] ---> | Object ref -> [value = 5] |

- **x** holds the value directly.
- **y** holds a reference to an object *that contains* the value.

7 Summary

Concept	Primitives	Objects
Type example	int, double, boolean	String, Integer, ArrayList
Stored in	Stack	Heap
Methods	None	Many
Speed	Faster	Slower
Object-oriented?	 No	 Yes

 In short:

Java is a fully object-oriented language — except for its **primitive types**, which are **not objects** but simple, built-in data types added for **performance efficiency**.

To work with them as objects, Java provides **wrapper classes** and **autoboxing**.



Step 1: The Context — “Write Once, Run Anywhere”

Java was designed with this motto:



“Write Once, Run Anywhere.”

That means:

- A Java program compiled on **Windows**, **Linux**, or **Mac** should behave **exactly the same**.
- The same code should give the **same output** everywhere — regardless of CPU type, OS, or hardware.

To make that possible, Java designers had to **standardize** how data types behave.



Step 2: What Happens in Other Languages (like C or C++)

In C or C++, data type sizes (like `int`, `long`, `short`) **depend on the machine and compiler**.

For example:

Type	16-bit Machine	32-bit Machine	64-bit Machine
------	----------------	----------------	----------------

<code>int</code>	2 bytes	4 bytes	4 or 8 bytes
------------------	---------	---------	--------------

long 4 bytes 4 bytes 8 bytes
g

So, the same code:

```
int x = 2147483647;
```

might work on one machine but **overflow** on another, because `int` could mean different sizes!

That breaks portability ❌



Step 3: How Java Solves It

Java says:

“No matter *where* you run your program — the data type sizes and ranges will be *exactly the same*.”

That’s what your book means by:

“Primitive types are defined to have an explicit range and mathematical behavior.”



Step 4: Java’s Fixed Type Sizes and Ranges

Type	Size (bits)	Range	Example
byte	8 bits	-128 to 127	byte b = 100;
short	16 bits	-32,768 to 32,767	short s = 5000;
int	32 bits	-2,147,483,648 to 2,147,483,647	int i = 100000;

<code>long</code>	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	<code>long l = 10000000000L;</code>
<code>float</code>	32 bits	~7 decimal digits	<code>float f = 3.14f;</code>
<code>double</code>	64 bits	~15 decimal digits	<code>double d = 3.14159265358979;</code>
<code>char</code>	16 bits	Unicode 0 to 65535	<code>char c = 'A';</code>
<code>boolean</code>	1 bit (conceptually)	<code>true</code> or <code>false</code>	<code>boolean flag = true;</code>

These **never change**, no matter the computer.

So an `int` is *always* 4 bytes (32 bits) — everywhere, every time 

Step 5: Why “Mathematical Behavior” is Explicit Too

Java also defines exactly **how arithmetic behaves** on primitives.

For example:

- Integer division always truncates toward zero.
- Overflow wraps around predictably.
- Floating-point arithmetic follows IEEE 754 standard.

Example:

```
int x = 2147483647; // max int
```

```
x = x + 1;
```

`System.out.println(x);` // prints -2147483648 (wraps around)

This will behave the *same* on every system, guaranteed.



Step 6: Why It Matters for Portability

In C/C++:

`int a = 10;` // may mean 2 bytes or 4 bytes

So if you send that integer over a network to another computer, it might not read it correctly (different byte size or endianness).

But in Java:

`int a = 10;` // always 4 bytes

Both sender and receiver (even on different OS or hardware) know exactly how many bits represent the data — ensuring perfect consistency. 🌍



Step 7: Summary

Concept	In C/C++	In Java
Data type size	Depends on compiler/system	Fixed and same everywhere
Range	Varies	Strictly defined
Math behavior	May differ between compilers	Standardized (IEEE 754, etc.)

Portability  Not guaranteed  Fully portable

Design goal Performance and flexibility Safety and portability

 In short:

Java defines each primitive type to have a **fixed size and behavior**, regardless of platform.

This ensures that your program runs **the same way on every machine**, fulfilling Java’s promise of portability — something C/C++ can’t guarantee.

Integers

Java defines four integer types: **byte**, **short**, **int**, and **long**. All of these are signed, positive and negative values. Java does not support unsigned, positive-only integers. Many other computer languages support both signed and unsigned integers. However, Java’s designers felt that unsigned integers were unnecessary. Specifically, they felt that the concept of *unsigned* was used mostly to specify the behavior of the *high-order bit*, which defines the *sign* of an integer value. As you will see in Chapter 4, Java manages the meaning of the high-order bit differently, by adding a special “unsigned right shift” operator. Thus, the need for an unsigned integer type was eliminated.

The *width* of an integer type should not be thought of as the amount of storage it consumes, but rather as the *behavior* it defines for variables and expressions of that type. The Java run-time environment is free to use whatever size it wants, as long as the types behave as you declared them. The width and ranges of these integer types vary widely, as shown in this table:

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

Step 1 — What Are “Unsigned Integers”?

Normally, when you store an integer in memory, it’s represented as **binary bits**.

For example, let’s take an 8-bit number (1 byte):

00000000 → 0

00000001 → 1

00000010 → 2

...

01111111 → 127

But what about the bit at the very front (the *highest-order bit*)?

It can either be treated as:

- a **sign bit** (for positive/negative distinction), or
- a **normal value bit** (for larger positive numbers only).

That's what distinguishes:

- **Signed integers** (can be negative)
- **Unsigned integers** (can't be negative, but can store larger positive numbers)

Step 2 — Signed vs Unsigned (Example)

Signed 8-bit integer (**byte** in Java)

- Range: **-128 to +127**

10000000 → -128

10000001 → -127

...

11111111 → -1

00000000 → 0

01111111 → 127

The **first bit (1)** means negative — it's called the **sign bit**.

Unsigned 8-bit integer (used in C/C++)

- Range: **0 to 255**

00000000 → 0

00000001 → 1

...

11111111 → 255

Here, the high-order bit is *not* a sign — it's just part of the value.



Step 3 — Why Java *Doesn't* Have Unsigned Types

Java's designers (James Gosling & team) decided:

“Unsigned integers are unnecessary and make arithmetic more confusing.”

Here's their reasoning:

1 Simplicity — One Type, One Rule

In C/C++ you have to worry about both:

```
int a;    // signed
```

```
unsigned b; // unsigned
```

Mixing them can lead to weird results:

```
int a = -1;
```

```
unsigned int b = 1;
```

```
if (a < b)
```

```
    printf("true");
```

else

```
printf("false");
```

This prints **✗ “false”** because when compared, **-1** is converted to an unsigned value → **4294967295!**

That’s confusing and error-prone.
Java wanted to avoid that entire headache.

2 Simpler Math Rules

In Java:

- All integers are **signed**.
- You can safely add, subtract, multiply, or divide without worrying about how the sign bit behaves.

No unexpected conversions or compiler-specific behavior.

3 Unsigned Values Are Rarely Needed

The designers observed:

“Unsigned integers are mostly used to manipulate bits — not real-world numbers.”

Example: In C, unsigned types are often used for low-level operations like memory addresses, bit masks, or raw binary data.

But Java wasn’t built for system-level programming — it runs inside the **JVM sandbox**, not directly on hardware.

So unsigned numbers weren’t that important.

Step 4 — But What If You *Do* Need Unsigned Behavior?

Java later added a few **helper methods** in classes like **Integer** and **Long** (since Java 8):

Example:

```
int x = Integer.parseUnsignedInt("4294967295"); // max unsigned int  
  
long y = Integer.toUnsignedLong(x);  
  
System.out.println(y); // prints 4294967295
```

You can also interpret signed numbers as unsigned when needed:

```
byte b = (byte) 255;  
  
System.out.println(Byte.toUnsignedInt(b)); // prints 255
```

So — while Java doesn't have separate unsigned types — it provides utilities to *treat* numbers as unsigned when required.

Step 5 — Visual Summary

Concept	Signed	Unsigned
Uses high-order bit for sign	✅ Yes	❌ No
Range (8-bit)	-128 → 127	0 → 255
Common in	Java, C, C++	C, C++
Java support	✅ Only signed	⚙️ Helper methods (since Java 8)
Reason for omission	Simplify arithmetic, avoid confusion	

Step 6 — The Textbook Line Explained Simply

“Java’s designers felt that unsigned integers were unnecessary. Specifically, they felt that the concept of unsigned was used mostly to specify the behavior of the high-order bit, which defines the sign of an integer value.”

Meaning:

- In languages like C, **unsigned** just changes how the topmost bit is *interpreted* (as part of the number instead of as a sign).
- Java’s designers saw no strong need for this — since Java already handles signed arithmetic safely and predictably.

In short:

Java does not have unsigned integer types because its creators wanted to keep the language simple, consistent, and less error-prone.

Unsigned types mostly affect how the **highest bit** is interpreted (as sign or value), and Java’s designers decided to always treat it as a sign bit — making all integer types **signed** by default.

Step 1 — What You Might Think It Means (and Why That’s Wrong)

At first glance, you might think:

“Oh, `int` is 4 bytes → so it always occupies 4 bytes in memory.”

But your book says:

“Don’t think of width as storage size — think of it as *behavior*.”

That’s because Java is **platform-independent** — you’re not supposed to care *how much memory* it uses internally, only *how it behaves* in your program.



Step 2 — In Simpler Words

- ♦ The *width* of a type (like `int` = 32 bits) defines **its range and arithmetic rules**,
not necessarily the exact number of bytes the JVM uses behind the scenes.

So “width” describes **logical behavior**, not **physical memory**.



Step 3 — Let’s See It Visually

Example:

```
int a = 10;
```

We say:

- `int` → 32-bit signed integer
- Range: -2,147,483,648 to +2,147,483,647
- Arithmetic: Wrap-around behavior on overflow

That’s its **behavior**.

Java *guarantees* this no matter where your code runs — Windows, Mac, Linux, or Android.

Now, inside your computer:

- The JVM may use 4 bytes of memory.
- Or it could optimize it differently internally (for registers, caching, or JIT compilation).

But **you don’t see that** — the observable behavior (range, math, type) remains identical.



Step 4 — Contrast with C or C++

In C:

```
int a = 10;
```

On a 16-bit compiler: `int` → 2 bytes

On a 32-bit compiler: `int` → 4 bytes

On a 64-bit compiler: could vary again!

So, its **behavior (range, overflow, promotion)** changes depending on the platform.

In Java:

```
int a = 10; // always 32-bit, behaves exactly the same everywhere
```

You can *rely* on it behaving consistently.

The underlying memory layout is JVM's problem — not yours.



Step 5 — Why It Matters: Portability & Abstraction

Java was designed to *hide machine-level details*.

The JVM abstracts the hardware and memory layer, so your focus is on **logic**, not **architecture**.

So you can safely assume:

```
int x = 10;
```

```
int y = 20;
```

```
int z = x + y; // always predictable math
```

— even though, under the hood, the JVM might be using CPU registers, caching, or compressed memory formats.

This is similar to how when you use a `String`, you don't worry about whether it's UTF-8, UTF-16, or ASCII in memory.

You just know its **behavior** (how it compares, concatenates, etc.).



Step 6 — The Key Phrase: “Behavior It Defines”

When the book says:

“The width of an integer type defines the behavior of variables and expressions of that type,”

it means:

- What range of values it can hold
- What happens on overflow
- What kind of arithmetic rules apply
- How type promotion works in expressions

That’s what *defines* the type — not the byte count in memory.

Step 7 — Example Showing Behavioral Focus

```
byte a = 10;    // 8 bits (-128 to 127)
```

```
short b = 30000; // 16 bits (-32768 to 32767)
```

```
int c = a + b;  // automatically promoted to int (32 bits)
```

Here, the “width” matters only because it defines **how** promotion and arithmetic happen — not how much RAM each consumes.

Analogy

Imagine a *cup* labeled “500 ml”.

You care that it **holds half a liter**, not what material it’s made of or how thick the walls are. That’s what Java’s “width defines behavior” means — it’s about **what it can hold**, not **how** it’s built internally.

Summary

Concept

Meaning

Width	Logical definition (bit count, range, arithmetic rules)
Storage	JVM's internal implementation — may vary
Programmer sees	Consistent behavior (same output everywhere)
Why Java does this	To make programs portable and abstracted from hardware

✓ In short:

When the book says “*width defines behavior, not storage*,” it means that Java guarantees how a data type behaves (its range and math rules) — but hides how much memory it actually uses.

You focus on *logic*, and the JVM handles *memory* behind the scenes to ensure portability and consistency.

byte

The smallest integer type is **byte**. This is a signed 8-bit type that has a range from −128 to 127. Variables of type **byte** are especially useful when you’re working with a stream of data from a network or file. They are also useful when you’re working with raw binary data that may not be directly compatible with Java’s other built-in types.

Although you might think that using a byte or short would be more efficient than using an int in situations in which the larger range of an int is not needed, this may not be the case. The reason is that when byte and short values are used in an expression, they are promoted to int when the expression is evaluated.



Step 1 — What the Book Is Saying

You might think:

“If I use **byte** or **short**, they’re smaller than **int**, so they should be faster or use less memory.”

That *sounds logical*, right?

But in reality, **Java automatically promotes `byte` and `short` to `int`** during arithmetic operations.

So using them doesn't make things faster — the JVM still does the math as `ints` internally.

Step 2 — Let's See It with an Example

```
byte a = 5;
```

```
byte b = 10;
```

```
byte c = a + b; // ❌ ERROR!
```

Wait — that's an error!

Even though `a` and `b` are both bytes, Java says:

"Possible lossy conversion from int to byte."

Why? Because of **type promotion**.

Step 3 — What Actually Happens Internally

When Java evaluates `a + b`,

- Both `a` and `b` (which are `bytes` = 8 bits)
- are **promoted to `int` (32 bits)**
- before the addition happens.

So this:

```
byte c = a + b;
```

is really doing:

```
int temp = (int)a + (int)b; // promoted!
```

```
byte c = (byte)temp;    // narrowing back to byte → possible data loss
```

That's why you get an error — because the result of the expression is an `int`, not a `byte`.

Step 4 — Why Java Does This

Because Java's designers wanted **all arithmetic operations to be done in at least 32 bits**, for safety and consistency.

In older CPUs (and in C), 8-bit and 16-bit operations were slower or less standardized, so Java decided to keep it simple:

Always do arithmetic using `int` (32-bit), unless you explicitly say otherwise.

So:

- `byte`, `short`, and `char` → promoted to `int` in expressions
 - `int` → promoted to `long` if needed
 - `float` → promoted to `double` if mixed with a double
-

Step 5 — Demonstrating Promotion in Code

Let's take an example:

```
byte a = 50;
```

```
byte b = 50;
```

```
byte c = (byte)(a + b);
```

```
System.out.println(c);
```

✓ Output: 100

Here, we had to explicitly cast `(byte)(a + b)` because:

- `a + b` is an `int`
 - assigning it directly to a `byte` would cause a compile-time error
-

Step 6 — It's Not Just for `byte`, Even `short` Works This Way

```
short x = 1000;
```

```
short y = 2000;
```

```
short z = (short)(x + y); // works with cast
```

Same rule applies — both are promoted to `int` before addition.

Step 7 — So Why the Book Says “May Not Be More Efficient”

Because:

- Even though `byte` and `short` occupy less memory *as variables*,
- when the CPU or JVM evaluates expressions, it uses `ints` anyway.
- So the “extra efficiency” you thought you’d get from smaller types doesn’t exist during computation.

In fact, on most modern CPUs, 32-bit arithmetic (`int`) is *faster* than 8-bit (`byte`), because CPUs are optimized for 32-bit or 64-bit operations.

Step 8 — Quick Summary Table

Type(s) in Expression	Promoted To	Example
byte + byte	int	(byte) (a + b)
short + short	int	(short) (x + y)
char + char	int	(char) (ch1 + ch2)
int + long	long	automatic
float + double	double	automatic

Analogy

Think of it like this 👉

If two people whisper (byte and short), but the translator (CPU) only listens in **normal voice** (int), the whispers are **amplified to normal voice** before translation.

So no matter how small your data types are, the processing happens at full “voice” level (i.e., int).

In Short

When you use byte or short in arithmetic, Java **automatically promotes** them to int before performing any operation.

Therefore, there's **no real performance gain** in using smaller types for calculations — they're only useful for conserving *memory storage*, not *CPU computation*.

The type float specifies a single-precision value that uses 32 bits of storage. Single precision is faster on some processors and takes half as much space as double precision, but will become imprecise when the values are either very large or very small. Single precision uses 32 bits (4 bytes) and double precision uses 64 bits (8 bytes) to store floating-point numbers. Double precision offers greater accuracy (around 15-17 decimal digits) and a much wider range of values compared to single precision (around 6-9 decimal digits), making it suitable for scientific calculations where precision is critical. Single precision is often preferred for applications like graphics programming where memory usage and speed are more important than extreme precision.

Feature	Single Precision	Double Precision
Memory	32 bits (4 bytes)	64 bits (8 bytes)
Exponent Bits	8 bits	11 bits
Mantissa Bits	23 bits	52 bits
Precision	~6-9 decimal digits	~15-17 decimal digits
Max Value	~3.4e+38	~1.8e+308
Best Use Case	Games, graphics, situations prioritizing speed and memory	Scientific, financial, and engineering calculations requiring high accuracy

Double precision, as denoted by the double keyword, uses 64 bits to store a value. Double precision is actually faster than single precision on some modern processors that have been optimized for high-speed mathematical calculations. All transcendental math functions, such as sin(), cos(), and sqrt(), return double values. When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, double is the best choice.

Characters

In Java, the data type used to store characters is **char**. However, C/C++ programmers beware: **char** in Java is not the same as **char** in C or C++. In C/C++, **char** is 8 bits wide. This is *not* the case in Java. Instead, Java uses *Unicode* to represent characters. Unicode defines a fully international character set that can represent all of the characters found in all human languages. It is a unification of dozens of character sets, such as Latin, Greek, Arabic, Cyrillic, Hebrew, Katakana, Hangul, and many more. At the time of Java's creation, Unicode required 16 bits. Thus, in Java **char** is a 16-bit type. The range of a **char** is 0 to 65,536. There are no negative **chars**. The standard set of characters known as ASCII still ranges from 0 to 127 as always, and the extended 8-bit character set, ISO-Latin-1, ranges from 0 to 255. Since Java is designed to allow programs to be written for worldwide use, it makes sense that it would use Unicode to represent characters. Of course, the use of Unicode is somewhat inefficient for languages such as English, German, Spanish, or French, whose characters can easily be contained within 8 bits. But such is the price that must be paid for global portability.

Excellent question 💡 — understanding **literals** in Java is super important because they're literally (pun intended 😊) the *raw values* that make up your code.

Let's break it down clearly and visually 🙌

What Are Literals?

In Java, a **literal** is a *constant, fixed value* that you write directly in your code — a value that **doesn't change during execution**.

They represent actual data **assigned to variables**.

For example:

```
int age = 21;    // 21 is a literal
```

```
char grade = 'A'; // 'A' is a literal
```

```
boolean isJavaFun = true; // true is a literal
```

```
String name = "Sheeba"; // "Sheeba" is a literal
```

Here:

- 21, 'A', true, "Sheeba" → all are **literals**.

- They directly represent the value stored in a variable.

The 6 Types of Literals in Java

Literal Type	Example	Explanation
Integer Literals	<code>10, -25, 0x1A, 0b1010</code>	Whole numbers — can be decimal, hexadecimal, or binary
Floating-Point Literals	<code>3.14, 2.5e3</code>	Numbers with decimals or exponent form
Character Literals	<code>'A', '\n', '\u0041'</code>	Single characters enclosed in single quotes
String Literals	<code>"Hello", "Java\nWorld"</code>	Sequence of characters enclosed in double quotes
Boolean Literals	<code>true, false</code>	Only two values — represent logical truth
Null Literal	<code>null</code>	Special literal for object references that point to nothing

1 Integer Literals

Integer literals represent whole numbers and can be written in **different number systems**:

Format	Example	Description
--------	---------	-------------

Decimal	10	Base 10
Hexadecimal	0x1A	Base 16 (prefix 0x or 0X)
Binary	0b1010	Base 2 (prefix 0b or 0B)
Octal	012	Base 8 (prefix 0)

👉 Example:

```
int dec = 10;    // decimal
```

```
int hex = 0xA;   // hexadecimal (10)
```

```
int bin = 0b1010; // binary (10)
```

```
int oct = 012;   // octal (10)
```

2 Floating-Point Literals

For decimal numbers or numbers with fractions:

```
double pi = 3.14159;
```

```
float price = 10.5f;    // must add 'f' or 'F' for float
```

```
double exp = 2.5e3;     // exponential notation →  $2.5 \times 10^3 = 2500.0$ 
```

3 Character Literals

Always enclosed in **single quotes** ' '

```
char grade = 'A';

char newline = '\n';    // escape sequence for new line

char unicodeA = '\u0041'; // Unicode for 'A'
```

👉 You can use:

- **Escape sequences** (`'\n'`, `'\t'`, `'\"'`, `'\\'`)
 - **Unicode representation** (`'\uXXXX'`)
-

4 String Literals

A sequence of characters enclosed in **double quotes** `" "`

```
String greeting = "Hello, World!";

String message = "Java\nRocks!"; // includes escape sequence
```

💡 Internally, a string literal creates a `String` object in the **String pool**.

5 Boolean Literals

They represent truth values:

```
boolean isJavaFun = true;

boolean isFishFlying = false;
```

Used in conditions, loops, and logical expressions.

6 The `null` Literal

`null` is a **special literal** that means “no object reference”.

```
String name = null; // name points to nothing
```

If you try to use it (like `name.length()`), you'll get a `NullPointerException`.



Bonus — Underscores in Numeric Literals

To make long numbers more readable, Java allows `_` between digits (since Java 7):

```
int bigNumber = 1_000_000; // same as 1000000
```

```
double price = 12_345.67_89; // same as 12345.6789
```



You **can't** start or end a literal with `_`.



Summary Table

Type	Example	Data Type
<code>10</code>	Integer Literal	<code>int</code>
<code>10L</code>	Long Literal	<code>long</code>
<code>3.14</code>	Floating-point Literal	<code>double</code>
<code>'A'</code>	Character Literal	<code>char</code>
<code>"Hello"</code>	String Literal	<code>String</code>

<code>true / false</code>	Boolean Literal	<code>boolean</code>
-------------------------------	-----------------	----------------------

<code>null</code>	Null Literal	—
-------------------	--------------	---

Visual Example

 Code |  Meaning

<code>int age = 20;</code>	20 → integer literal
<code>float pi = 3.14f;</code>	3.14f → float literal
<code>char ch = 'J';</code>	'J' → character literal
<code>String s = "Java";</code>	"Java" → string literal
<code>boolean b = true;</code>	true → boolean literal

In Simple Words

Literals are the **fixed values** you type directly into your Java program — they are the actual “data” part of your code, while variables are just “containers” that hold those values.

Java Literals — Valid & Invalid Notations

Literals = *fixed values* that you write directly in code.

Java allows **numeric**, **character**, **string**, **boolean**, and **null** literals — and each type has very specific **notation rules**.

1 Integer Literals (whole numbers)

Java supports **four bases** for integer literals:

Base	Prefix	Example	Meaning
Decimal	none	10	Ten
Binary	0b or 0B	0b1010	Ten
Octal	leading 0	012	Ten
Hexadecimal	0x or 0X	0xA	Ten

Valid Integer Literals

```
int a = 10;    // decimal
```

```
int b = 0b1010; // binary (Java 7+)
```

```
int c = 012;    // octal
```

```
int d = 0x1A;    // hexadecimal
```

```
int e = 1_000_000; // underscores allowed (for readability)
```

```
long f = 10L;    // long literal (L or l suffix)
```

Invalid Integer Literals

`int a = 08; // ❌ Invalid octal (8 not allowed in base-8)`
`int b = 0b102; // ❌ Invalid binary (2 not allowed in base-2)`
`int c = 0xG1; // ❌ Invalid hex (G not valid)`
`int d = _1000; // ❌ Cannot start with underscore`
`int e = 1000_; // ❌ Cannot end with underscore`
`int f = 1__000; // ✅ Allowed (double underscores are fine, in middle)`

💡 Suffixes

Suffix	Meaning
--------	---------

L or l	long literal (e.g. 123L)
--------	--------------------------

⚠️ Use **uppercase L** — lowercase **l** looks like the number 1.

💧 2 Floating-Point Literals

Used for **decimal** or **exponential** numbers.

✅ Valid Floating Literals

`double a = 3.14; // double literal`
`float b = 3.14f; // float literal (needs f or F)`
`double c = 1e3; // exponential notation ( $1 \times 10^3 = 1000.0$ )`
`float d = 2.5E-3f; // exponential float ( $2.5 \times 10^{-3} = 0.0025$ )`
`double e = 1_000.25; // underscores allowed`

❌ Invalid Floating Literals

`float a = 3.14;` `//` ❌ default is double, needs suffix `f` or `F`

`double b = 1.2e;` `//` ❌ missing exponent value

`double c = .e3;` `//` ❌ must have a digit before or after `'.'`

`double d = 3._14;` `//` ❌ underscore cannot be adjacent to dot

`double e = 3.14_F;` `//` ❌ underscore cannot be before suffix

💡 Suffixes

Suffix	Meaning
--------	---------

<code>f</code> or <code>F</code>	float literal
----------------------------------	---------------

<code>d</code> or <code>D</code>	double literal (optional)
----------------------------------	---------------------------

abc 3 Character Literals

Represent a **single 16-bit Unicode character**, in single quotes `' '`.

✅ Valid Character Literals

`char a = 'A';`

`char b = '9';`

`char c = '\n';` `//` newline escape sequence

`char d = '\u0041';` `//` Unicode for `'A'`

`char e = '"';` `//` single quote character

`char f = '\\';` `//` backslash

✗ Invalid Character Literals

`char a = 'AB';` // ✗ only one character allowed

`char b = '';` // ✗ cannot be empty

`char c = ' ';` // ✓ space is allowed though

`char d = 'hello';` // ✗ more than one character

`char e = 'A\n';` // ✗ escape sequence counts as one, but extra char invalid

No, a `char` variable cannot be assigned empty quotes (`''`). A `char` type is designed to store a single character. Empty quotes represent a string literal with no characters, which is a different data type (a string).

In languages like C, C++, or Java, a `char` variable must always hold a valid character. If you need to represent the absence of a character or a "blank" character, you typically use:

- The null character `\0`: This represents a character with a numerical value of 0 and is often used as a string terminator.

C



```
char myChar = '\0';
```

- A space character `' '`: If you intend to represent a visible blank space.

Java



```
char myChar = ' ';
```

Attempting to assign empty quotes to a `char` will result in a compile-time error, typically a "Empty character literal" error.

Example:

Java



```
char myChar = ''; // This will cause a compile-time error
```

Alternatives for representing a "blank" or "empty" character:

- Null Character (`\0` or `\u0000`): This represents the null character, which has a numerical value of 0. It is often used to mark the end of a character sequence or to signify an "empty" state for a character.

Java



```
char myChar = '\0';
```

- `null` for `Character` objects: While a primitive `char` cannot be `null`, its wrapper class `Character` can be assigned `null`. This is useful when working with collections or situations where an absence of a character value needs to be explicitly represented.

Java



```
Character myCharacter = null;
```

4 String Literals

Strings are sequences of characters enclosed in **double quotes** `" "`.

Valid String Literals

String a = "Hello";

```
String b = "Java\nRocks";  
  
String c = "He said, \"Hi!\"";  
  
String d = "Path: C:\\Java\\bin";  
  
String e = "";    // empty string
```

Invalid String Literals

```
String a = 'Hello';    //  must use double quotes  
  
String b = "Hello;    //  missing closing quote  
  
String c = "Hi"World"; //  cannot concatenate like this
```

5 Boolean Literals

Only **two possible values**: **true** or **false**.

Valid

```
boolean x = true;  
  
boolean y = false;
```

Invalid

```
boolean x = True;    //  case-sensitive  
  
boolean y = 0;       //  integers not allowed  
  
boolean z = "true";  //  string not boolean
```

6 Null Literal

`null` represents *no object reference*.

✓ Valid

String name = null;

Integer num = null;

✗ Invalid

int n = null; // ✗ primitive types cannot be null

boolean flag = null; // ✗ same reason

🧠 7 Underscore Rules in Numeric Literals (Java 7+)

✓ Allowed

✗ Not Allowed

1_000_000 _1000 (start)

0b1010_10 0b_1010 (right after
10 prefix)

3.14_15 3._1415 (adjacent to
dot)

1_2_3_4 1_2_3_ (end)

Underscores are purely **for readability**, ignored by compiler.

8 Summary Table

Literal Type	Valid Example	Invalid Example	Notes
Integer	<code>0b1010</code> , <code>0x1F</code> , <code>012</code> , <code>1_000</code>	<code>0b102</code> , <code>_10</code> , <code>10_</code>	L suffix → long
Floating	<code>3.14f</code> , <code>1e3</code> , <code>1_000.5</code>	<code>3._14</code> , <code>.e2</code>	f/F for float
Char	<code>'A'</code> , <code>'\n'</code> , <code>'\u0041'</code>	<code>'AB'</code> , <code>''</code>	Single quotes only
String	<code>"Hello"</code> , <code>"A\nB"</code>	<code>'Hello'</code> , <code>"Hi</code>	Double quotes only
Boolean	<code>true</code> , <code>false</code>	<code>True</code> , <code>0</code>	Case-sensitive
Null	<code>null</code>	<code>Null</code> , <code>NULL</code>	Only for objects

9 Visualization Summary

 Code

 Meaning

`int x = 0xA;` Hexadecimal literal (10)

`int y = 0b1010;` Binary literal (10)

`float z = 3.14F;` Float literal

`char ch = '\u0041';` Unicode for 'A'

`String s = "Java";` String literal

`boolean b = true;` Boolean literal

String n = null; Null literal

✓ In short:

Literals are *the fixed, directly written values* in Java code. Each literal type follows strict rules — especially numeric bases, suffixes, and underscore placement — to maintain Java's precision and portability.

long literals (the essentials)

- **Type:** 64-bit signed integer.
- **Range:** `-9,223,372,036,854,775,808` to `9,223,372,036,854,775,807`.
- **Syntax:** add suffix `L` or `l` to an integer literal to make it a `long`.
 - Prefer **uppercase L** because lowercase `l` looks like `1`.

✓ Valid long literals

```
long a = 12345678901L;    // decimal long
long b = 0x7FFFFFFFFFFFFFL; // hex long (note the L)
long c = 0b1010_1010L;    // binary with underscores
long d = 01234567L;       // octal long (leading zero)
long e = 1_000_000_000L;  // underscores OK for readability
```

✗ Invalid or problematic cases

```
long x = 12345678901; // ✗ default is int -> overflow compile error if out of int range
long y = 0xFFG1L;     // ✗ invalid hex digit G
long z = _1000L;       // ✗ underscore at start not allowed
long u = 1000_L;      // ✗ underscore adjacent to suffix not allowed
```


Important notes

- **Integer literals without suffix are `int` by default.** If the number is larger than `int` range, you **must** add `L` (otherwise compile error).
- You can use binary (`0b...`), octal (leading `0`), and hex (`0x...`) forms with an `L` suffix.
- Underscores are allowed in numeric literals (Java 7+), but **not** at start/end, next to `0x`, `0b`, decimal point, or suffix.

Wrapper class & unsigned helpers

- Wrapper: `Long` (useful in collections: `List<Long>`).
 - Java provides **unsigned** helpers (e.g. `Long.toUnsignedString`, `Long.divideUnsigned`) to treat `long` values as unsigned when needed.
-

The variable is the basic unit of storage in a Java program. A variable is defined by the combination of an identifier, a type, and an optional initializer. In addition, all variables have a scope, which defines their visibility, and a lifetime.

A block defines a scope. Thus, each time you start a new block, you are creating a new scope. A scope determines what objects are visible to other parts of your program. It also determines the lifetime of those objects.

As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope. Thus, when you declare a variable within a scope, you are localizing that variable and protecting it from unauthorized access and/or modification. Indeed, the scope rules provide the foundation for encapsulation. Scopes can be nested. For example, each time you create a block of code, you are creating a new, nested scope. When this occurs, the outer scope encloses the inner scope. This means that objects declared in the outer scope will be visible to code within the inner scope. However, the reverse is not true. Objects declared within the inner scope will not be visible outside it.

1 First — what is scope?

Scope means **where in the program a variable, method, or class is visible or accessible**.

It defines the *lifetime* and *visibility* of a name (variable, method, class, etc.).

Example:

```
{  
  
    int x = 10; // x exists only inside this block  
  
}
```

System.out.println(x); //  error: x is out of scope

2 Traditional Scopes in Other Languages

In older languages like **C**, **C++**, or **JavaScript (pre-ES6)**, we typically have:

Scope type	Description	Example
Global Scope	Variable accessible everywhere in the program	<code>int g = 10;</code> outside all functions
Local Scope	Variable accessible only inside a function or block	inside a function <code>{ int x; }</code>

So in those languages, you can define variables that are truly *global* — visible to all functions and modules.

3 Java's Philosophy — “No Real Global Scope”

Java is **strictly object-oriented**.

Everything (except primitives) lives **inside a class**.

That means Java doesn't have **true global variables or functions** like C/C++.

For example:

// In C:

```
int count = 0; // global variable accessible anywhere
```

// In Java:

```
public class Example {  
    static int count = 0; // this is class-level (not truly global)  
}
```

You can *simulate* global variables by using **public static** fields, but those are still bound to a **class**, not floating freely.

That's what the textbook means by:

"It is possible to create what amounts to being a global scope, but it is by far the exception, not the rule."

✅ You *can* do it with **public static** fields, but you *shouldn't* — it breaks encapsulation and OOP discipline.

4 The Two Major Scopes in Java

So, instead of global vs local, Java defines **two main categories of scope**:

Scope Type	Description	Where Defined	Example
Class Scope	Members (fields, methods) defined inside a class but outside any method	visible throughout the class	static or instance variables

Method Scope	Variables defined inside a method or block	visible only inside that method/block	local variables, parameters
---------------------	--	---------------------------------------	-----------------------------

Example

```
public class Example {

    // ----- Class Scope -----

    static int staticCounter = 0; // belongs to class

    int instanceCounter = 0;      // belongs to each object

    public void increment() {

        // ----- Method Scope -----

        int localCount = 0;      // only inside this method

        staticCounter++;

        instanceCounter++;

        localCount++;

        System.out.println(localCount);

    }

    public static void main(String[] args) {

        Example obj = new Example();

        obj.increment();

    }
```

```
}
```

Explanation:

Variable	Scope	Visible Where	Lifetime
<code>staticCounter</code>	Class scope	All methods of class	As long as class is loaded
<code>instanceCounter</code>	Class scope (instance-level)	All instance methods	As long as object exists
<code>localCount</code>	Method scope	Only within <code>increment()</code>	Created & destroyed each call

5 Why the Book Says the Distinction Is “Somewhat Artificial”

Because in **object-oriented Java**, everything lives inside a **class**, even methods — so method scope is *nested inside* class scope.

So technically, there’s only *one big scope* (the class) containing smaller ones (methods). But since methods have very different rules (locals, parameters, no `static`, etc.), it’s *useful* to talk about them separately.

That’s what this line means:

“Even this distinction is somewhat artificial. However, since the class scope has several unique properties ... this distinction makes sense.”

6 Visual Summary

Program

└─ Class Scope (Example)

└─ Static fields → shared by all objects

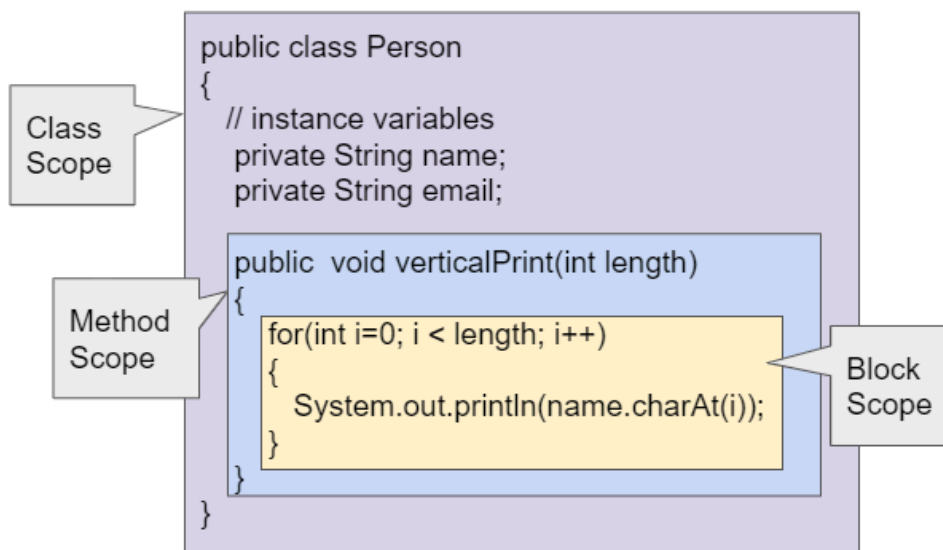
└─ Instance fields → each object has its own copy

└─ Methods

└─ Method Scope (local variables, parameters)

✓ In short:

- In older languages, variables could exist globally.
 - In Java, everything is wrapped inside a class → no true global scope.
 - The two important scopes are:
 - **Class scope** → static & instance members.
 - **Method scope** → local variables & parameters.
 - The separation is *conceptual*, but it helps understand variable lifetimes and access.
-



Let's break down **how** `public static` fields can *simulate* a “global variable” in Java — and *why* that's discouraged in real-world object-oriented programming.

1 What “Global” Means

In languages like C:

```
int counter = 0; // global variable
```

That variable can be accessed *from anywhere* in the program, by any function, file, or module.

It truly lives in one shared memory space.

Java does **not** have true global variables. Everything lives inside a **class**.

2 Simulating Global Variables with `public static`

If you declare a `public static` field, you're effectively creating one shared variable for the entire program.

Example

```
public class GlobalData {  
    public static int counter = 0; // acts like a "global" variable  
}
```

Now, any other class can access it directly:

```
public class Main {  
    public static void main(String[] args) {  
        GlobalData.counter++; // modify it  
        System.out.println(GlobalData.counter); // read it  
    }  
}
```

✓ Effect:

- `GlobalData.counter` exists as soon as the class is loaded.
- Every class and object in the JVM shares the same variable.
- You don't need an object of `GlobalData` to access it.

Hence, it *behaves like* a global variable.

🧠 3 Why This Works

Because:

- `static` → belongs to the **class**, not to any object.
- `public` → accessible from **anywhere** in the program.

So, a `public static` field = **one shared, always-accessible variable**.

🚫 4 Why It's Discouraged in OOP

Java was designed to enforce **encapsulation** —
each class should control access to its internal data.

But a `public static` variable breaks that rule because:

Problem	Example
✗ No control over changes	Any code anywhere can modify it.
✗ Tight coupling	Many classes depend directly on the same data.

✗ Hard to debug

When multiple threads or classes modify it, it's unclear who changed it.

✗ Breaks encapsulation

The internal state of a class is directly exposed.

✓ 5 The Recommended OOP Way

Instead of using a **public static** variable directly, use **accessor methods** (getters/setters) or **singleton classes** if you really need global-like access.

Example: Using **private static** + **Getter/Setter**

```
public class Config {  
  
    private static int counter = 0;  
  
  
    public static int getCounter() {  
  
        return counter;  
  
    }  
  
  
    public static void incrementCounter() {  
  
        counter++;  
  
    }  
  
}
```

Now other classes must access it in a controlled way:

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Config.incrementCounter();  
  
    }  
  
}
```

```
        System.out.println(Config.getCounter());
    }
}
```

- ✓ Safer.
- ✓ Still shared.
- ✓ Follows OOP encapsulation.

6 Summary Table

Concept	Keyword(s)	Shared by	Accessible from	Typical Use
Instance variable	none	Each object	Only through that object	Object state
Static variable	<code>static</code>	All objects of a class	Through class name	Shared resources
Public static variable	<code>public static</code>	All objects and all classes	Anywhere	Acts like “global” variable (not recommended)

Analogy

Imagine your whole program as a building:

- Each class is a **room**.
- Instance variables are **items inside the room** — private to that room.
- Static variables are **items placed in a shared hallway** — everyone in the building can use them.

- Public static variables are **items left outside on the street** — literally *anyone* can mess with them.

That's why Java discourages them — they're accessible everywhere, but unsafe for good design.

Here is another important point to remember: variables are created when their scope is entered, and destroyed when their scope is left. This means that a variable will not hold its value once it has gone out of scope. Therefore, variables declared within a method will not hold their values between calls to that method. Also, a variable declared within a block will lose its value when the block is left. Thus, the lifetime of a variable is confined to its scope. If a variable declaration includes an initializer, then that variable will be reinitialized each time the block in which it is declared is entered.

If the two types are compatible, then Java will perform the conversion automatically. For example, it is always possible to assign an int value to a long variable. However, not all types are compatible, and thus, not all type conversions are implicitly allowed. For instance, there is no automatic conversion defined from double to byte. To obtain a conversion between incompatible types, you must use a cast, which performs an explicit conversion between incompatible types.

When one type of data is assigned to another type of variable, an automatic type conversion will take place if the following two conditions are met:

- The two types are compatible.
- The destination type is larger than the source type.

When these two conditions are met, a widening conversion takes place. For example, the int type is always large enough to hold all valid byte values, so no explicit cast statement is required.

Casting Incompatible Types

Although the automatic type conversions are helpful, they will not fulfill all needs. For example, what if you want to assign an **int** value to a **byte** variable? This conversion will not be performed automatically, because a **byte** is smaller than an **int**. This kind of conversion is sometimes called a *narrowing conversion*, since you are explicitly making the value narrower so that it will fit into the target type.

To create a conversion between two incompatible types, you must use a cast. A *cast* is simply an explicit type conversion. It has this general form:

(target-type) value

If the integer's value is larger than the range of a byte, it will be reduced modulo (the remainder of an integer division by the) byte's range. A different type of conversion will occur when a floating-point value is assigned to an integer type: truncation. As you know, integers do not have fractional components. Thus, when a floating-point value is assigned to an integer type, the fractional component is lost. For example, if the value 1.23 is assigned to an integer, the resulting value will simply be 1. The 0.23 will have been truncated. Of course, if the size of the whole number component is too large to fit into the target integer type, then that value will be reduced modulo the target type's range.

```
byte a = 40;  
byte b = 50;  
byte c = 100;  
int d = a * b / c;
```

The result of the intermediate term **a * b** easily exceeds the range of either of its **byte** operands. To handle this kind of problem, Java automatically promotes each **byte**, **short**, or **char** operand to **int** when evaluating an expression. This means that the subexpression **a*b** is performed using integers—not bytes. Thus, 2,000, the result of the intermediate expression, **50 * 40**, is legal even though **a** and **b** are both specified as type **byte**.

As useful as the automatic promotions are, they can cause confusing compile-time errors. For example, this seemingly correct code causes a problem:

```
byte b = 50;  
b = b * 2; // Error! Cannot assign an int to a byte!
```

The code is attempting to store **50 * 2**, a perfectly valid **byte** value, back into a **byte** variable. However, because the operands were automatically promoted to **int** when the expression was evaluated, the result has also been promoted to **int**. Thus, the result of the expression is now of type **int**, which cannot be assigned to a **byte** without the use of a cast. This is true even if, as in this particular case, the value being assigned would still fit in the target type.

In cases where you understand the consequences of overflow, you should use an explicit cast, such as

```
byte b = 50;  
b = (byte) (b * 2);
```

which yields the correct value of 100.

The Type Promotion Rules

Java defines several *type promotion* rules that apply to expressions. They are as follows: First, all **byte**, **short**, and **char** values are promoted to **int**, as just described. Then, if one operand is a **long**, the whole expression is promoted to **long**. If one operand is a **float**, the entire expression is promoted to **float**. If any of the operands are **double**, the result is **double**.

Here, the square brackets follow the type specifier, and not the name of the array variable. For example, the following two declarations are equivalent:

```
int a1[] = new int[3];  
int[] a2 = new int[3];
```

The following declarations are also equivalent:

```
char twod1[][] = new char[3][4];  
char[][] twod2 = new char[3][4];
```

This alternative declaration form offers convenience when declaring several arrays at the same time. For example,

```
int[] nums, nums2, nums3; // create three arrays
```

creates three array variables of type **int**. It is the same as writing

```
int nums[], nums2[], nums3[]; // create three arrays
```

The alternative declaration form is also useful when specifying an array as a return type for a method. Both forms are used in this book.

Java's string type, called `String`, is not a primitive type. Nor is it simply an array of characters. Rather, `String` defines an object, and a full description of it requires an understanding of several object-related features.

Java does not support or allow pointers. (Or more properly, Java does not support pointers that can be accessed and/or modified by the programmer.) Java cannot allow pointers, because doing so would allow Java programs to breach the firewall between the Java execution environment and the host computer. (Remember, a pointer can be given any address in memory—even addresses that might be outside the Java run-time system.) Since C/C++ make extensive use of pointers, you might be thinking that their loss is a significant disadvantage to Java. However, this is not true. Java is designed in such a way that as long as you stay within the confines of the execution environment, you will never need to use a pointer, nor would there be any benefit in using one.

Starting with Java SE 10, you can use the `var` type identifier to declare a local variable. In doing so, you let the compiler decide what is the real type of the variable you create. Once created, this type cannot be changed. `var` type identifier.

1. You can only use it for *local variables* declared in methods, constructors and initializer blocks.
2. `var` cannot be used for fields, nor for method or constructor parameters.
3. The compiler must be able to choose a type when the variable is declared. Since `null` has no type, the variable must have an initializer.

1. What is `var` in Java (from Java 10)?

`var` is **NOT** a datatype like in JavaScript or Python.

`var` is **just a compiler feature** that says:

“Hey compiler, YOU infer the variable’s type from the value I assign right now.”

Example:

```
var x = 10;
```

Compiler sees `10` → `int`, so it becomes:

```
int x = 10;
```

💡 After inference, **x is an `int` forever**.
You cannot assign another type later.

2. Why does the book say “Once created, this type cannot be changed”?

Because this:

```
var x = 10; // x is int
```

```
x = "hello"; // ❌ ERROR
```

The type was inferred **once**, and it's final.

3. Where can **var** be used?

Only in **local variables**, inside:

- methods
- constructors
- initializer blocks

✓ Allowed:

```
void test() {  
    var count = 5;  
    var text = "Hello";  
}
```

❌ NOT allowed:

❌ For fields (instance variables)

```
class A {  
    var x = 10; // ❌ compile error  
}
```

❌ For method parameters

```
void test(var value) { } // ❌
```

✗ For return types

```
public var getValue() {} // ✗
```

Why?

Because **var** is **not a real type**.

It's only a compiler shortcut for local inference.

⚠ 4. Why must **var** have an initializer?

Because the compiler must know the type **right now**.

Correct:

```
var x = 10; // compiler decides: int
```

This does NOT compile:

```
var x; // ✗ Error: cannot infer type
```

And this also fails:

```
var x = null; // ✗ null has NO type, cannot infer
```

But this works:

```
String y = null; // because you've explicitly given the type
```

💣 5. Why can't you assign null?

Because **null** does not carry type information.

- `"hello"` → `String`
- `10` → `int`
- `true` → `boolean`
- `new ArrayList<>()` → `ArrayList`

But:

- `null` → **no type** 😞

So:

`var x = null; // ❌` compiler has no clue what type you want

6. Real Examples (to understand deeply)

✓ Good use:

```
var list = new ArrayList<String>();  
  
// list is inferred as ArrayList<String>
```

✓ Good use (loops):

```
for (var item : list) { ... }
```

❌ BAD use (ambiguous):

```
var a = 1;    // int  
  
var b = 1.0;  // double  
  
var c = 1f;   // float
```

Makes code less readable.

✓ **Good with generics:**

```
var map = new HashMap<String, List<Integer>>();
```

7. Why did Java add **var**?

To reduce verbosity:

Before Java 10:

```
Map<String, List<Integer>> map = new HashMap<String, List<Integer>>();
```

After Java 10:

```
var map = new HashMap<String, List<Integer>>();
```

Cleaner and easier, especially for long generic types.

Summary in Simple English

Concept	Meaning
var is not a type	It is a hint to the compiler to infer type
Needs initializer	Because compiler must infer type immediately

Cannot be null

Null has no type

Cannot be used in fields/method parameters

Because inference only works for locals

Once inferred, type is fixed

Cannot assign another type later

1 What is the output?

```
byte b = 127;
```

```
b++;
```

```
System.out.println(b);
```

✓ Answer: **-128**

💡 Why tricky:

- **byte** range = **-128 to 127**
- Incrementing 127 causes **overflow**, wrapping to -128.

2 Is this valid?

```
byte x = 10;
```

```
byte y = 20;
```

```
byte z = x + y;
```

✗ **Answer: No, compilation error.**

💡 **Why:**

- Arithmetic on smaller types (`byte`, `short`, `char`) is **promoted to `int`**.
- So `x + y` becomes `int`, cannot assign to `byte` without cast.

Correct:

```
byte z = (byte)(x + y);
```

3 What is the output?

```
char c = 'A';
```

```
c = c + 1;
```

```
System.out.println(c);
```

✗ **Compilation error.**

💡 **Why:**

- `c + 1` → promoted to `int`
- Cannot assign `int` to `char` without cast.

Correct:

```
c = (char) (c + 1); // prints 'B'
```

4 Does this compile?

```
char c = 65;
```

```
System.out.println(c);
```

✓ Yes → prints: A

💡 Why:

- `char` can take an **integer literal** if it fits 0–65535.
-

5 What is the output?

```
int i = 1;
```

```
long l = 1;
```

```
System.out.println(i == l);
```

✓ true

💡 Why:

- `i` is promoted to long
 - Values equal → true.
-

6 What is the output?

```
float f = 1.0;
```

```
System.out.println(f);
```

✗ Compilation error.

💡 Why:

- By default floating literals = **double**
- Double cannot be assigned to float.

Correct:

```
float f = 1.0f;
```

7 What is the output?

```
System.out.println(10 / 4);
```

```
System.out.println(10 / 4.0);
```

✓ **Output:**

2

2.5

💡 **Why:**

- First uses **integer division**
 - Second promotes to **double division**
-

8 What is the output?

```
double d = 0.1 + 0.2;
```

```
System.out.println(d == 0.3);
```

✗ **false**

 Why:

- Floating point precision errors.
 - $0.1 + 0.2 \neq 0.3$ exactly in IEEE-754.
-

9 What happens?

```
long l = 1_00_00_000;  
int i = l;
```

 **Compilation error.**

 Why:

- Narrowing conversion from long $\rightarrow$ int requires explicit cast.

Correct:

```
int i = (int) l;
```

10 Will this compile?

```
int a = 10;  
int b = 010;  
System.out.println(b);
```

 **In modern Java (≥ 8): Yes, prints 8**

 Why tricky:

- `010` is **octal literal**, not decimal
(leading zero → octal)
 - `010` (base 8) = **8** (decimal)
-

1 What is the output?

```
byte b = -128;
```

```
b = (byte) (b / -1);
```

```
System.out.println(b);
```

✅ **Answer: -128**

💡 **Why this is evil:**

- $-128 / -1 = 128$ mathematically.
 - But 128 **cannot be represented** in a byte.
 - Overflow wraps it back to **-128**.
-

2 What does this print?

```
char c = '\u0000';
```

```
System.out.println(c == 0);
```

✅ **true**

💡 **Why:**

- `'\u0000'` is char value `0`
 - `0` is an **int literal**
 - char converted → int → comparison → true.
-

3 What happens?

```
int i = 1;
```

```
Integer j = 1;
```

```
System.out.println(i == j);
```

✓ **true**

💡 **Why:**

- `j` is unboxed to int → `1 == 1`.

BUT...

4 What about this one?

```
Integer a = 128;
```

```
Integer b = 128;
```

```
System.out.println(a == b);
```

✗ **false**

💡 **Why tricky:**

- Integer caching works only for values **-128 to 127**

- 128 is **not cached**, so different objects.
-

5 What is the output?

```
double d1 = Double.NaN;  
double d2 = Double.NaN;  
System.out.println(d1 == d2);
```

 **false**

 **Why:**

- NaN is not equal to anything, not even itself.
-

6 Will this compile? What is the output?

```
short s = 1;  
s += 1;  
System.out.println(s);
```

 **Compiles**

Output → **2**

 **Why:**

- `s += 1` includes an **implicit cast**
- equivalent to `s = (short)(s + 1)`
- But `s = s + 1` alone causes compile error.

7 What is the output?

```
char c = 65535;  
  
c++;  
  
System.out.println((int) c);
```

Output:

0

💡 Why:

- Max char = 65535
- Increment wraps to **0** (like overflow).

8 What is the output?

```
long x = 10;  
  
float y = x;  
  
System.out.println(y);
```

Output:

10.0

💡 Why tricky:

- long → float is **widening**, so no cast required
(People mistakenly think float < long because float < long in storage size.)

9 What is printed?

```
int a = Integer.MAX_VALUE;
```

```
int b = a + 1;
```

```
System.out.println(b < a);
```

Output:

true

💡 Why:

- Overflow: `MAX_VALUE + 1` → `Integer.MIN_VALUE`
- So `b < a` becomes `-2147483648 < 2147483647` → true.

10 What is the output?

```
System.out.println(0.0 == -0.0);
```

```
System.out.println(Double.compare(0.0, -0.0));
```

Output:

true

1

💡 Why:

- `0.0 == -0.0` → true (same value)

- But IEEE-754 considers them **different** representations
→ `Double.compare` returns **1** if first > second.
-